NetworkX Library

BSc. Nhat-Tan Bui

Software Engineering Lab (SELab),
University of Science - VNUHCM
Artificial Intelligence and Computer Vision Lab (AICV),
University of Arkansas, Fayetteville
https://tanbuinhat.github.io/

Content

- Introduction
- Basic functions
- Graph traversals
- Girvan-Newman algorithm

NetworkX

- Documentation: https://networkx.guide/getting-started
- Python library for working with the structure, dynamics and functions of complex networks.
- NetworkX also provides popular group of algorithms including graph traversals, centrality, shortest path and community detection.
- Installing
  - pip install networkx

Basic functions

- Explore dataset

```
data = pd.read_csv('data.csv')
data
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...
795	719	Diancie	Rock	Fairy	600	50	100	150	100	150	50	6	True
796	719	DiancieMega Diancie	Rock	Fairy	700	50	160	110	160	110	110	6	True
797	720	HoopaHoopa Confined	Psychic	Ghost	600	80	110	60	150	130	70	6	True
798	720	HoopaHoopa Unbound	Psychic	Dark	680	80	160	60	170	130	80	6	True
799	721	Volcanion	Fire	Water	600	80	110	120	130	90	70	6	True

800 rows × 13 columns

Basic functions

- Read a graph from Pandas Dataframe.

```
g = nx.Graph()
g = nx.from_pandas_edgelist(data, source='Name', target='Type 1')

#source is the column in dataframe indicates source nodes, target is the
column in dataframe indicates target nodes

print(g)
```

Basic functions

- Visualize a graph

```
def draw_sub_graph(sub_graph):
    subgraph = g.subgraph(sub_graph)
    colors = []
    for node in g:
        if node in data["Type 1"].values:
            colors.append("lightblue")
        else:
            colors.append("lightgreen")

    nx.draw(subgraph, node_size=1000, with_labels=True, font_weight='bold',
  node_color=colors)

    plt.show()

draw_sub_graph(g)
```

Basic functions

- Add nodes and edges

```
g.add_nodes_from(
    [
        ("Knowledge", {"label": "Type 1", "name": "Knowledge Type"}),
        ("Windy", {"label": "Type 1", "name": "Windy Type"})
    ]
) # adding multiple nodes

g.add_edge("Knowledge", "Windy", label="NEW_IN") # adding one edge

g.remove_nodes_from(["Knowledge", "Windy"]) # remove multiple nodes

g.remove_edges_from([("Knowledge", "Windy")]) # remove one edge
```

Graph traversals

- Breadth-first search (BFS): This algorithm is used to explore graph data by starting from graph's root and visiting the nearest node based on distance from the source node.

```
pos = nx.planar_layout(g) #position nodes without edge intersections.

bfs = nx.bfs_tree(g, source="Grass") #source is the starting node for
searching
nx.draw(bfs, pos, with_labels=True)

plt.show()
```

Graph traversals

- Depth-first search (DFS): This algorithm is used to explore graph data by starting from graph's root and visting each neighboring node as far as possible before backtracking.

```
pos = nx.planar_layout(g) #position nodes without edge intersections.

bfs = nx.dfs_tree(g, source="Grass") #source is the starting node for
searching
nx.draw(bfs, pos, with_labels=True)

plt.show()
```

Girvan-Newman algorithm

This algorithm is used to detect community structure by iteratively removing the edge with the highest betweenness (number of the shortest paths).

```
communities = girvan_newman(g)
node_groups = []
for com in next(communities):
    node_groups.append(list(com))

print(node_groups)
```